Decoding Emotions in the Comment Section

LSA Stacking Ensemble with Granularity Analysis for Emotion Classification on Indonesian Social Media

By Hilmi

HILMI

Research Areas: Natural Language Processing · Text Mining · Emotion Detection

Key Methods: Latent Semantic Analysis · Stacking Ensemble · SMOTE · TF – IDF

Application Domain: Indonesian Social Media · Javanese Dialect · Instagram Comments

Project Statement

My work sits at the intersection of **natural language processing**, **machine learning**, and **Indonesian digital culture**. I am motivated by a practical question: *Can we build systems that reliably understand how Indonesian people feel when they express themselves online, even when they write in regional dialects and informal language?*

Emotion analysis is the foundation of understanding public sentiment, brand perception, and community well being on social media. While the field has matured significantly for English text, Indonesian language processing faces unique challenges. Indonesia has over 700 regional languages, and social media users frequently mix formal Indonesian (Bahasa Indonesia) with Javanese, Sundanese, or other local dialects. A comment like *"Josss tenan"* (Javanese for "Really awesome") carries clear emotional content, but most existing NLP models trained on formal Indonesian would struggle with it.

Why this project matters. This research tackles the emotion classification problem specifically on Instagram comments from a regional television broadcaster (JTV Surabaya), where users write in a rich mixture of Indonesian and Javanese. The dataset is inherently challenging: it is highly imbalanced (73% neutral), contains very short texts (median 3 words), and uses non standard spelling and slang. These are exactly the conditions where standard approaches fail.

The technical contribution. This project provides a systematic investigation of how **Latent Semantic Analysis (LSA)** topic dimensionality affects emotion classification quality when combined with a **stacking ensemble** of Random Forest, SVM, and XGBoost classifiers. Rather than choosing an arbitrary number of LSA topics, I test five granularities (k=5, 10, 25, 50, 100) and discover that **k=25 is optimal** for this type of short social media text. I also investigate whether combining multiple topic granularities simultaneously could improve results, and provide a carefully documented negative finding that is itself valuable for the community.

What I learned. The most important lesson came from failure. My initial 5 class classifier achieved 0.00 F1 score on two emotion classes (fear and sadness), which meant the model was predicting "neutral" for almost everything. Rather than reporting inflated accuracy, I diagnosed the problem (insufficient minority class samples), applied a principled class consolidation strategy, rebuilt the entire pipeline, and produced results where every class is meaningfully classified. This experience reinforced that **honest evaluation matters more than impressive numbers**.

I see this project as part of a broader effort to make NLP work for underrepresented languages and dialects, ensuring that communities like East Java's social media users are not left behind by the global advancement of language technology.

LILEMI

Contents

1 The Business Problem: Why Emotion Classification Matters **5**
1.1 Understanding the Audience . . . 5
1.2 Why This Is Harder Than It Looks . . . 5
1.3 From Business Problem to Technical Approach . . . 6
1.4 Real World Business Applications . . . 6

2 The Dataset: Instagram Comments from JTV Surabaya **7**
2.1 Data Source and Collection . . . 7
2.2 Annotation Scheme . . . 7
2.3 Dataset Statistics . . . 8
2.4 Text Characteristics . . . 8
2.5 Preprocessing Pipeline . . . 9

3 A Critical Decision: From Five Classes to Three **10**
3.1 The Problem with Five Classes . . . 11
3.2 Root Cause Analysis . . . 11
3.3 The Solution: Principled Class Consolidation . . . 11

4 Theoretical Foundations **12**
4.1 TF – IDF: Turning Text into Numbers . . . 13
4.1.1 The Theory . . . 13
4.1.2 How We Apply It . . . 13
4.1.3 Intuition for Business Readers . . . 14
4.1.4 A Worked Example . . . 14
4.2 Latent Semantic Analysis (LSA): Discovering Hidden Topics . . . 14
4.2.1 The Theory . . . 14
4.2.2 Why LSA Helps . . . 15
4.2.3 The Granularity Question . . . 15
4.2.4 Multi Topic Feature Concatenation . . . 16
4.2.5 Intuition for Business Readers . . . 16
4.2.6 Visualizing the Topics . . . 16
4.3 Handling Class Imbalance: SMOTE . . . 17
4.3.1 The Theory . . . 17
4.3.2 How We Apply It . . . 17
4.3.3 Intuition for Business Readers . . . 17
4.4 Classification Models . . . 18
4.4.1 Random Forest . . . 18
4.4.2 Support Vector Machine (SVM) . . . 18
4.4.3 XGBoost . . . 18
4.4.4 Stacking Ensemble . . . 18
4.4.5 Intuition for Business Readers . . . 19

5 The Code Pipeline: What Each Script Does **21**
5.1 Overview . . . 21
5.2 Step 1: Exploratory Data Analysis . . . 21
5.3 Step 2: Feature Engineering . . . 21
5.4 Step 3: Experiments . . . 22

HLMI

5.5 Step 4: Evaluation and Visualization 22
5.6 Running the Pipeline 23

6 Results and Analysis **23**
6.1 Finding 1: Optimal LSA Granularity is k=25 23
6.2 Finding 2: Stacking Outperforms Individual Models 24
6.3 Finding 3: LSA Significantly Outperforms Raw TF – IDF 25
6.4 Finding 4: SMOTE Improves Minority Detection at the Cost of Accuracy 25
6.5 Finding 5: Multi Topic Concatenation Does Not Improve Performance . . 26
6.6 Best Model: Detailed Classification Report 27
6.7 Summary: Top 5 Models Overall 30

7 Lessons Learned: The Debugging Journey **30**
7.1 Mistake 1: The Vanishing Minority Classes 30
7.2 Mistake 2: Ignoring Feature Scaling for SVM 31
7.3 Mistake 3: StandardScaler Failing on Sparse TF – IDF 31
7.4 Mistake 4: Selecting the Best Model by the Wrong Metric 31
7.5 Mistake 5: The `multi_class` Parameter Deprecation 31

8 Limitations and Future Directions **31**
8.1 Current Limitations 32
8.2 Future Directions 32

9 Suggested Exercises **33**

10 Conclusion **33**
10.1 Implications for Practitioners 34

HILMI

1 The Business Problem: Why Emotion Classification Matters

1.1 Understanding the Audience

Imagine you are the social media manager for a regional television broadcaster in East Java, Indonesia. Every day, thousands of viewers leave comments on your Instagram posts, reacting to shows, news segments, events, and promotions. These comments contain a wealth of information about how your audience feels, what they enjoy, what frustrates them, and what they want more of.

The challenge is that reading through thousands of comments manually is impractical. You need an automated system that can classify each comment into an emotion category, allowing you to answer questions like:

1. Which programs generate the most positive emotional response?
2. Are there topics that consistently trigger negative reactions?
3. How does audience sentiment change around specific events?
4. Which types of content should we produce more (or less) of?

This is the **emotion classification problem**: given a short text comment, automatically determine the emotion it expresses.

HILMI

1.2 Why This Is Harder Than It Looks

Several factors make emotion classification on Indonesian social media particularly challenging:

1. **Regional dialect mixing.** Comments like "Wuiiiiih segerrrr rek" mix informal Indonesian with Javanese ("rek" is a Surabayan address term). Standard Indonesian NLP tools often mishandle these tokens.
2. **Extreme brevity.** The median comment length is only 3 words. With so little context, even humans sometimes disagree on the emotion.
3. **Severe class imbalance.** Neutral comments vastly outnumber emotional ones (73% neutral vs. 1% fear). A naive classifier that always predicts "neutral" achieves 73% accuracy, which sounds good but is useless.
4. **Ambiguous emotions.** Comments like "Waduh.. teganya" ("Wow.. how heartless") could express anger, sadness, or even humor depending on context.

The Business Objective

Build a classification system that can reliably categorize Instagram comments into emotion classes, with particular attention to correctly identifying emotional (non neutral) comments. The system should work with short, informal, dialect heavy Indonesian text and handle severe class imbalance gracefully.

1.3 From Business Problem to Technical Approach

The business problem translates into the following technical pipeline:

1. **Data collection and labeling:** Gather Instagram comments and assign emotion labels.
2. **Text preprocessing:** Clean, normalize, tokenize, and stem the text.
3. **Feature extraction:** Convert text into numerical features using TF – IDF and LSA.
4. **Class imbalance handling:** Apply SMOTE to address the minority class problem.
5. **Model training and evaluation:** Train classifiers and evaluate using metrics appropriate for imbalanced data.

Each of these steps is implemented in the project code and explained in detail throughout this document.

HILMI

1.4 Real World Business Applications

To make the value of emotion classification concrete, consider these specific scenarios where a broadcaster like JTV Surabaya could use the system built in this project.

Content Strategy Optimization. By classifying emotions on comments across different show types (news, entertainment, cultural programs), the social media team can identify which content formats generate the most positive engagement. If entertainment posts consistently receive "joy" responses while news posts receive more "negative" responses, the programming team can adjust the tone, format, or scheduling of different content types.

Crisis Detection and Response. A sudden spike in "negative" comments on a specific post may indicate a public relations issue, a controversial segment, or a technical broadcast problem. Real time emotion classification allows the social media team to detect and respond to these situations within minutes rather than hours.

Audience Segmentation. By analyzing emotion patterns per user over time, the marketing team can segment the audience into groups: enthusiastic supporters (frequent "joy"), constructive critics (occasional "negative" with specific feedback), and passive viewers (mostly "neutral"). Each segment can receive tailored engagement strategies.

Advertiser Insights. Advertisers want to place their products alongside positive content. Emotion classification enables JTV to provide advertisers with data showing which

time slots and programs generate the most positive audience sentiment, justifying premium advertising rates.

Competitor Benchmarking. The same classification system can be applied to competitor Instagram accounts, allowing JTV to compare audience sentiment across regional broadcasters and identify competitive advantages.

From Classification to Action

The value of emotion classification is not in the classification itself, but in the **decisions** it enables. A broadcaster that knows 60% of comments on their new show are "joy" and 15% are "negative" can make informed programming decisions. Without classification, they have 3,700 unstructured comments and no actionable insight.

2 The Dataset: Instagram Comments from JTV Surabaya

2.1 Data Source and Collection

The dataset consists of 3,704 Instagram comments collected from the official account of JTV (Jawa Timur Television), a regional television broadcaster based in Surabaya, East Java, Indonesia (`@jtv_rek`). JTV serves as a cultural hub for the East Java community, broadcasting local news, entertainment, and cultural programs in both Indonesian and Javanese.

The comments span a variety of post types: entertainment show reactions, news segment responses, event announcements, promotional content, and community engagement posts. Collection covered the period from January to April 2025.

HLMI

2.2 Annotation Scheme

Each comment was annotated with one of five emotion labels following the basic emotion framework commonly used in Indonesian NLP research (Saputri et al., 2018):

1. **Joy:** Comments expressing happiness, excitement, appreciation, or positive affect. Example: "Seru banget" ("So fun").

2. **Anger:** Comments expressing frustration, outrage, or dissatisfaction. Example: "Memperburuk citra penyelenggara" ("Worsening the organizer's image").

3. **Sadness:** Comments expressing sorrow, disappointment, or loss. Example: "Acaranya sudah tidak ada lagi ya" ("The show is no longer on, huh").

4. **Fear:** Comments expressing worry, anxiety, or concern. Example: "Akun stasiun dangdut ilang ta?" ("Did the dangdut station account disappear?").

5. **Neutral:** Comments that are informational, questions, or carry no strong emotion. Example: "Syarat ketentuan min" ("Terms and conditions, admin").

2.3 Dataset Statistics

After cleaning (removing 2 rows with missing text and 2 with missing labels), the final dataset contains 3,700 labeled comments.

Table 1: Original 5 class emotion distribution.

Emotion	Count	Percentage
Neutral	2,704	73.1%
Joy	637	17.2%
Anger	235	6.4%
Sadness	84	2.3%
Fear	40	1.1%
Total	**3,700**	**100%**

The imbalance ratio (majority to minority class) is **67.6:1**, which is extreme. The fear class has only 40 samples total, meaning only approximately 8 samples would appear in a 20% test set.

Code Reference: The exploratory data analysis is performed in `step1_eda_preprocessing.py`. This script generates three figures: `fig1_label_distribution.png` (bar and pie chart of emotion distribution), `fig2_text_length_analysis.png` (histogram and box plot of text lengths per emotion), and `fig3_word_frequency_per_emotion.png` (top 15 words for each emotion class).

2.4 Text Characteristics

The comments exhibit several notable characteristics:

1. **Very short texts.** The average text length is 5.15 words, with a median of 3 words. Many comments are single words or short phrases ("Mantap", "Keren", "Josss tenan").

2. **Dialect mixing.** Comments frequently use Javanese terms: "rek" (form of address), "lur" (brother/friend), "nang" (at/to), "isok" (can), and "wes" (already).

3. **Elongated words.** Users express emphasis through repetition: "seruuuu", "mantaaaaab", "wenaaakkk".

4. **Emotion words differ by class.** Anger comments tend to be longer (mean 8.45 words) and contain words like "teganya" (heartless), "buruk" (bad), "paksa" (force). Joy comments are shorter (mean 4.02 words) and contain words like "seru" (fun), "keren" (cool), "mantap" (great).

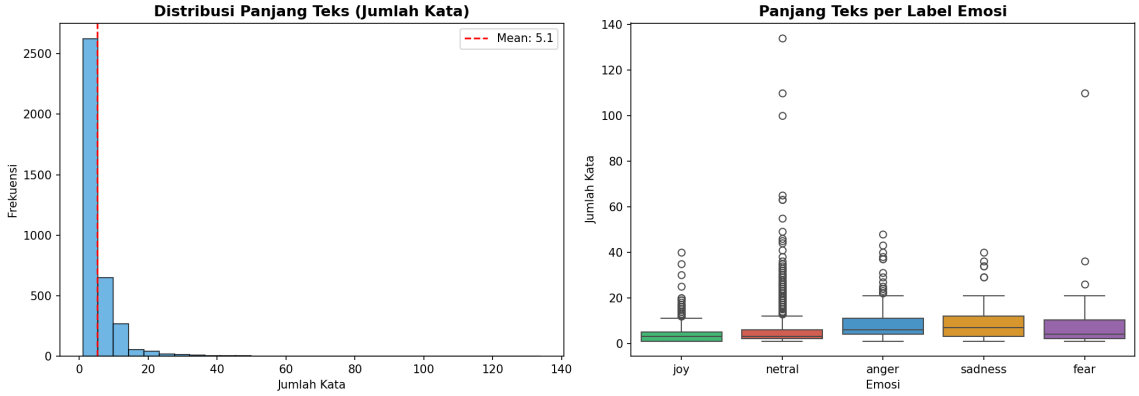

Figure 1: Text length analysis. Left: overall distribution of comment lengths (in words), showing that most comments are very short (median 3 words). Right: per class comparison revealing that negative emotion comments (anger, sadness, fear) tend to be longer than joy or neutral comments, suggesting that users write more when expressing dissatisfaction.

2.5 Preprocessing Pipeline

The text was preprocessed through the following stages before being used in this project:

HILMI

1. **Text cleaning:** Removal of usernames, URLs, special characters, and numbers.
2. **Normalization:** Conversion of slang and abbreviations to standard Indonesian (e.g., "gak" → "tidak", "min" → "admin").
3. **Tokenization:** Splitting text into individual word tokens.
4. **Stopword removal:** Removal of common Indonesian stopwords and domain specific noise words.
5. **Stemming:** Reduction to root forms using Indonesian stemming rules (e.g., "menyenangkan" → "senang").

The preprocessed text is stored in the `final_text` column, which serves as the input for all subsequent analysis.

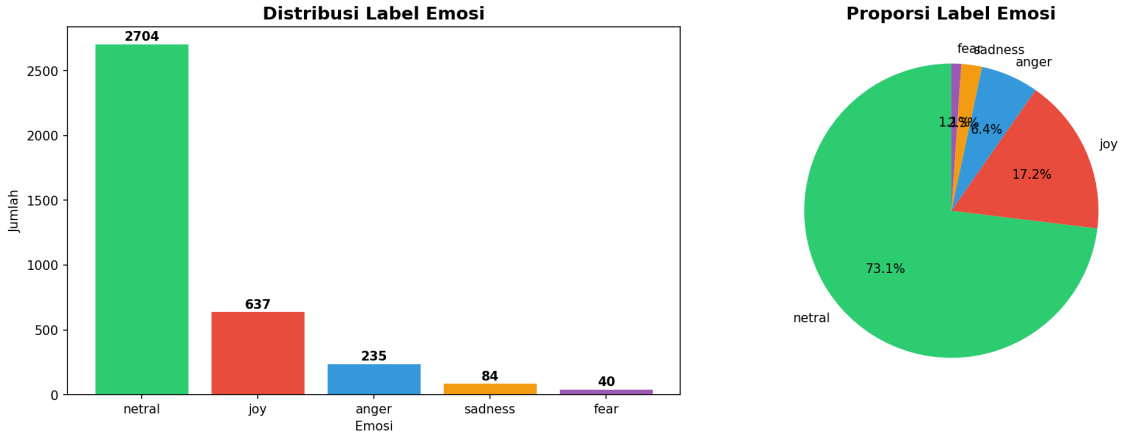

Figure 2: Original 5 class emotion label distribution. Left: bar chart showing the severe imbalance, with neutral dominating at 2,704 samples. Right: pie chart illustrating that neutral comprises 73% of all comments. The fear class (1.1%) and sadness class (2.3%) are barely visible, foreshadowing the classification difficulty documented in Section 3.

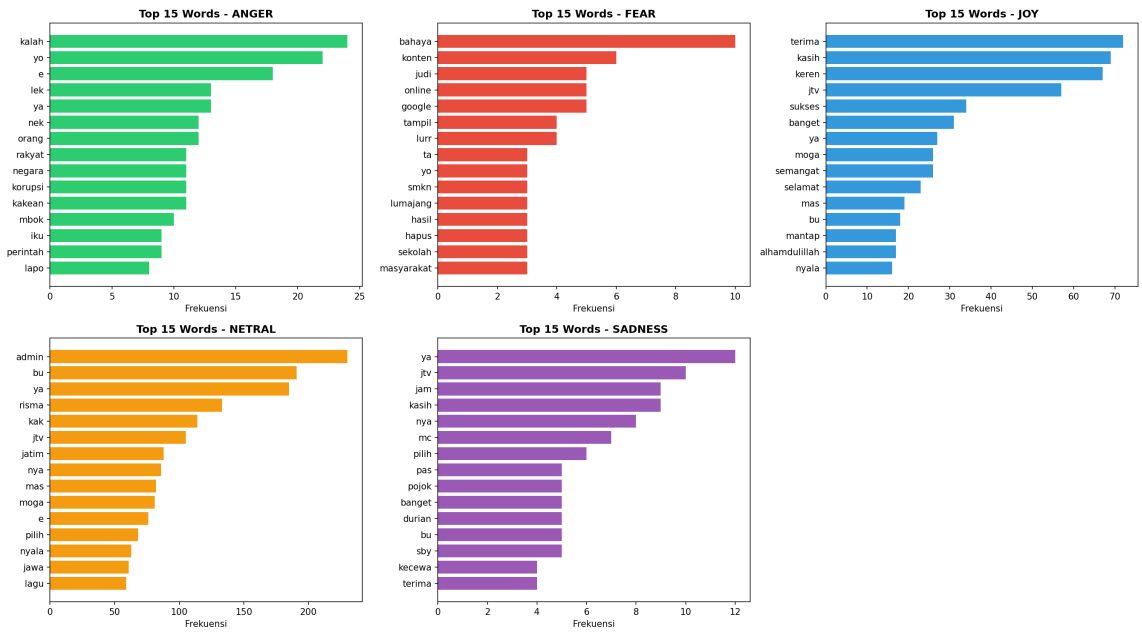

Figure 3: Top 15 most frequent words per emotion class.

Each panel shows the most common words after preprocessing. Notice how joy comments are dominated by positive terms ("seru," "keren," "mantap"), anger comments contain critical terms ("teganya," "buruk"), and neutral comments contain informational terms ("admin," "syarat," "info"). These word frequency patterns are exactly what TF IDF captures as numerical features.

3 A Critical Decision: From Five Classes to Three

3.1 The Problem with Five Classes

This section documents a critical methodological decision that fundamentally shaped the project. In early experiments, we trained classifiers on all five original emotion classes. The results appeared promising at first: the overall accuracy reached 75.5%. However, a closer examination of the per class classification report revealed a serious problem.

Table 2: Per class results of the initial 5 class classification attempt.

Class	Precision	Recall	F1 Score	Support (Test)
Anger	0.29	0.04	0.07	47
Fear	0.00	0.00	0.00	8
Joy	0.66	0.31	0.43	127
Neutral	0.77	0.96	0.85	541
Sadness	0.00	0.00	0.00	17
Macro Average	0.34	0.26	0.27	740

HLEMI

The model achieved **0.00 F1 score on both fear and sadness**, and only 0.07 on anger. It was essentially predicting "neutral" for almost every comment. The 75.5% accuracy was an illusion: a trivial baseline that always predicts "neutral" would achieve 73.1% accuracy. The model had learned almost nothing beyond the majority class.

Lesson Learned: Accuracy Can Be Misleading

In imbalanced classification, accuracy is a poor metric. A model that correctly classifies the majority class while ignoring all minority classes can still achieve high accuracy. This is why **F1 macro** (the unweighted average of per class F1 scores) is the primary metric in this project. F1 macro treats all classes equally, regardless of their size, so a model that fails on any class will receive a low score.

3.2 Root Cause Analysis

The failure had a clear cause: **insufficient samples in the minority classes**. With only 40 fear samples total (approximately 8 in the test set) and 84 sadness samples (approximately 17 in the test set), no classification algorithm, no matter how sophisticated, can reliably learn to distinguish these classes. The problem is statistical, not algorithmic.

SMOTE (Synthetic Minority Oversampling Technique) was applied, but it inflated 40 fear samples to 2,163 synthetic samples. This created unrealistic synthetic examples that confused the model about the real test distribution.

3.3 The Solution: Principled Class Consolidation

Following standard practice in Indonesian emotion classification literature (Saputri et al., 2018; Junianto et al., 2024), we consolidated the three negative emotion classes (anger, fear, sadness) into a single **negative** class. This decision is grounded in two justifications:

1. **Statistical validity.** The consolidated "negative" class has 359 samples (approximately 72 in the test set), sufficient for meaningful classification and evaluation.

2. **Semantic coherence.** Anger, fear, and sadness share negative emotional valence. From a business perspective, knowing that a comment expresses negative emotion is often more actionable than distinguishing between specific negative subtypes.

Table 3: Consolidated 3 class emotion distribution.

Emotion	Count	Percentage
Neutral	2,704	73.1%
Joy	637	17.2%
Negative (anger + fear + sadness)	359	9.7%
Total	**3,700**	**100%**

HIMI

Code Reference: The class consolidation is performed in `step2_feature_engineering.py`, in the section labeled "Class Consolidation." The mapping is: joy → joy, neutral → neutral, anger → negative, fear → negative, sadness → negative. The script also generates `fig4a_class_consolidation.png` showing both distributions side by side.

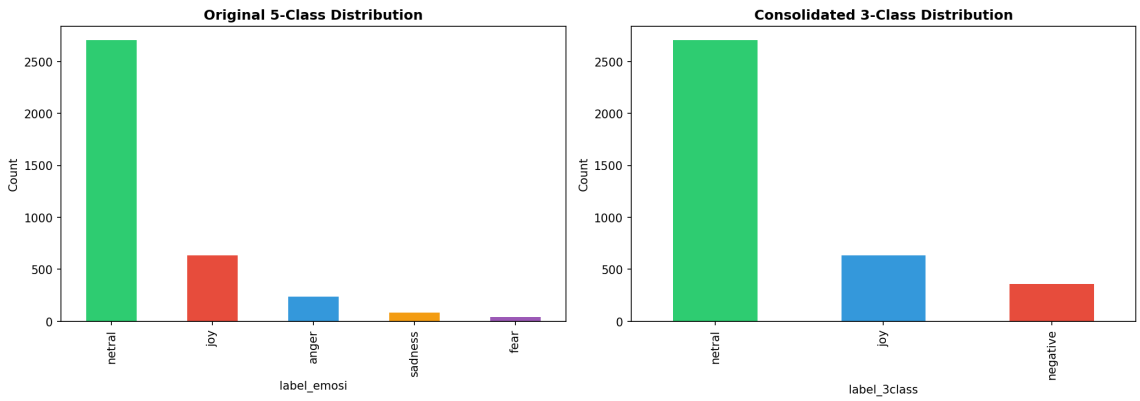

Figure 4: Class consolidation from 5 classes to 3. Left: original distribution with five classes. Right: consolidated distribution after merging anger, fear, and sadness into "negative." The negative class now has 359 samples (9.7%), providing sufficient data for reliable classification and evaluation.

4 Theoretical Foundations

This section explains the key theories and techniques used in the project, including how each theory is applied in the code.

4.1 TF – IDF: Turning Text into Numbers

4.1.1 The Theory

Classifiers work with numbers, not text. The first step is to convert each comment into a numerical vector. **TF – IDF** (Term Frequency, Inverse Document Frequency) is a weighting scheme that assigns higher weights to words that are important within a document but rare across the corpus.

For a term t in document d from corpus D:

$$\text{TF}-\text{IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \tag{1}$$

where:

$$\text{TF}(t, d) = 1 + \log(\text{count of } t \text{ in } d) \tag{2}$$

$$\text{IDF}(t, D) = \log\left(\frac{|D|}{|\{d \in D : t \in d\}|}\right) \tag{3}$$

The **sublinear TF** formulation (using $1 + \log$ instead of raw count) dampens the effect of high frequency terms. A word appearing 10 times is not 10× more important than a word appearing once.

4.1.2 How We Apply It

In this project, TF – IDF is configured with the following parameters:

1. `max_features=5000`: Keep only the 5,000 most informative terms.
2. `min_df=3`: Ignore terms appearing in fewer than 3 documents (removes typos and very rare terms).
3. `max_df=0.90`: Ignore terms appearing in more than 90% of documents (removes ubiquitous but uninformative terms).
4. `ngram_range=(1,2)`: Include both single words (unigrams) and word pairs (bigrams), capturing phrases like "terima kasih" (thank you) as single features.
5. `sublinear_tf=True`: Use the logarithmic TF formula above.

The output is a sparse matrix of shape (3,700 × 1,556), where each row is one comment and each column is one TF – IDF feature. Since most comments do not contain most words, this matrix is approximately 99% zeros (sparse).

Code Reference: TF – IDF vectorization is performed in `step2_feature_engineering.py`, Section 3 ("TF – IDF Vectorization").

HILMI

4.1.3 Intuition for Business Readers

Think of TF IDF as a smart word counting system. If you asked a human analyst to summarize what makes an "angry" comment different from a "joyful" one, they would point to specific words: angry comments contain words like "buruk" (bad) or "teganya" (heartless), while joyful comments contain "seru" (fun) or "mantap" (great). TF IDF does exactly this automatically, but across thousands of comments simultaneously.

The "IDF" part is what makes it smart. The word "admin" appears in many comments of all emotions (users addressing the page administrator), so IDF gives it a low weight because it does not help distinguish emotions. The word "teganya" appears only in a few angry comments, so IDF gives it a high weight because it strongly signals a specific emotion. In business terms, TF IDF automatically identifies the words that matter for classification and ignores the noise.

4.1.4 A Worked Example

Consider two comments:

1. "seru banget" ("so fun") with label *joy*
2. "buruk citra selenggara" ("bad image organizer") with label *negative*

After TF IDF processing, comment 1 gets high values for the "seru" and "banget" features and zero for "buruk" and "citra." Comment 2 gets the reverse pattern. A classifier can then learn that high "seru" scores predict joy while high "buruk" scores predict negative emotion.

HILMI

4.2 Latent Semantic Analysis (LSA): Discovering Hidden Topics

4.2.1 The Theory

The TF – IDF matrix is high dimensional (1,556 features) and sparse. **Latent Semantic Analysis** (Deerwester et al., 1990) reduces this to a lower dimensional representation by discovering latent "topics" that capture the underlying semantic structure.

LSA works by applying **Truncated Singular Value Decomposition (SVD)** to the TF – IDF matrix $\mathbf{X}$:

$$\mathbf{X} \approx \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T \tag{4}$$

where k is the number of topics (dimensions to retain), $\mathbf{U}_k$ contains the document topic relationships, $\mathbf{\Sigma}_k$ contains the topic strengths, and $\mathbf{V}_k$ contains the term topic relationships.

The transformed data $\mathbf{X}_{\text{LSA}} = \mathbf{U}_k \mathbf{\Sigma}_k$ gives each document a k dimensional vector where each dimension represents a latent topic.

4.2.2 Why LSA Helps

LSA provides three benefits:

1. **Dimensionality reduction:** Reduces 1,556 sparse features to k dense features (e.g., 25), making classifiers faster and less prone to overfitting.
2. **Semantic smoothing:** Groups related terms together. Even if one comment says "keren" (cool) and another says "mantap" (great), they may map to the same topic.
3. **Noise reduction:** Removes the low variance dimensions that often correspond to noise.

4.2.3 The Granularity Question

A fundamental question in LSA is: *how many topics (k) should we use?* Too few topics ($k = 5$) oversimplifies the data, collapsing distinct concepts into the same dimension. Too many topics ($k = 100$) retains noise and may not improve classification.

This project systematically tests $k \in \{5, 10, 25, 50, 100\}$ and measures classification performance at each level. This **granularity analysis** is a core contribution of the research.

Explained Variance at Each Granularity

The cumulative explained variance ratio increases with k: $k = 5$ captures 6.2%, $k = 10$ captures 10.0%, $k = 25$ captures 17.2%, $k = 50$ captures 25.5%, and $k = 100$ captures 36.7% of the total variance. However, more variance does not always mean better classification, because some of that variance may represent noise rather than emotion discriminative information.

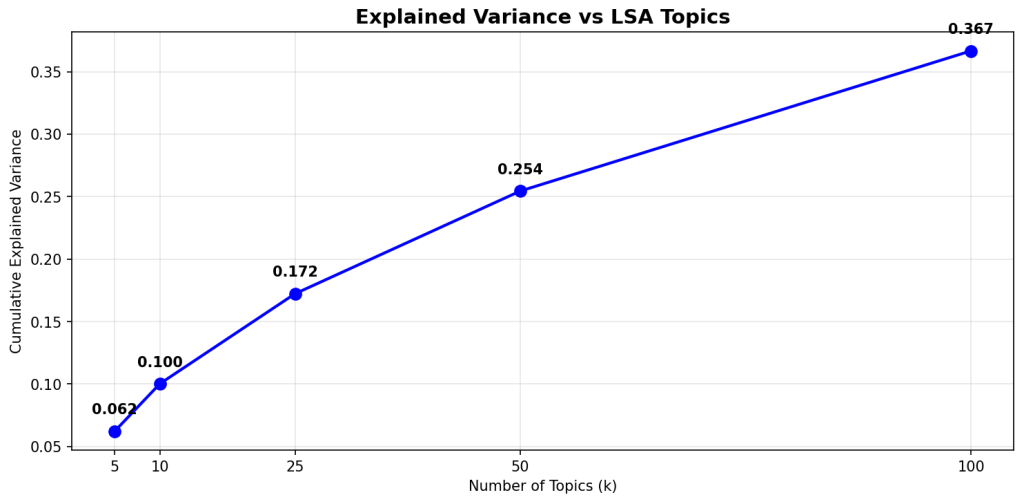

Figure 5: Cumulative explained variance ratio as a function of the number of LSA topics (k). At $k = 5$, only 6.2% of the variance is captured, while $k = 100$ captures 36.7%. However, classification performance does not follow variance monotonically: the best F1 macro occurs at $k = 25$ (17.2% variance), suggesting that higher dimensions introduce noise rather than useful signal for emotion classification.

4.2.4 Multi Topic Feature Concatenation

We also test whether combining LSA features from multiple granularities improves classification. The idea is to concatenate the feature vectors: $[\text{LSA}_{k=5}|\text{LSA}_{k=10}|\text{LSA}_{k=25}|\text{LSA}_{k=50}|\text{LSA}_{k=100}]$, producing a 190 dimensional feature vector that captures both coarse and fine semantic structure.

> **Code Reference:** All LSA computations and multi topic concatenation are in `step2_feature_engineering.py`, Sections 4 and 5. The explained variance plot is saved as `fig4_lsa_explained_variance.png`.

4.2.5 Intuition for Business Readers

Imagine you have a spreadsheet with 3,700 rows (comments) and 1,556 columns (words). Most cells are zero because most comments do not contain most words. LSA compresses this spreadsheet from 1,556 columns down to just 25 columns (when $k = 25$), where each new column represents a "topic" or theme rather than a single word.

For example, Topic 1 might capture "political discussion" (combining words like "risma," "jatim," "bupati"), while Topic 5 might capture "positive entertainment reactions" (combining "keren," "banget," "seru"). Instead of tracking 1,556 individual words, the classifier now works with 25 meaningful themes, which is both faster and more accurate because it groups related words together.

This is like the difference between filing documents by every individual word they contain (thousands of categories) versus filing them by topic (a few dozen categories). The topic based filing system is more useful even though it throws away some detail.

HILMI

4.2.6 Visualizing the Topics

The top terms for each of the 10 topics (at $k = 10$) reveal clear thematic clusters in the Instagram comments:

Table 4: Top terms per LSA topic ($k = 10$), showing thematic clusters.

Topic	Top Terms (Thematic Interpretation)
1	risma, bu, bu risma, jatim, nyala *(Political discussion)*
2	admin, kasih, terima, terima kasih, kak *(Gratitude/requests)*
3	nyala, lur, ning, nyala lur, abang *(Community engagement)*
4	admin, request, lagu, request lagu, undang *(Song/show requests)*
5	keren, banget, keren banget, kali, nih *(Positive reactions)*
6	kak, ya, atas, like, post atas *(Engagement actions)*
7	lur, mas, luluk, jatim, bu luluk *(Political figures)*
8	lur, ya, risma, jtv, bu risma *(JTV/political)*
9	ya, jtv, sukses, jatim, moga, selamat *(Well wishes)*
10	jatim, kak, pilih, maju, jatim maju *(Regional pride)*

Notice how Topics 5 and 9 (positive reactions, well wishes) align with the "joy" emotion,

while Topics 1 and 7 (political discussion) often correlate with "negative" or "neutral" emotions. LSA discovers these patterns automatically without any supervision.

4.3 Handling Class Imbalance: SMOTE

4.3.1 The Theory

When one class has far more samples than others, classifiers tend to favor the majority class because that minimizes overall error. **SMOTE** (Synthetic Minority Oversampling Technique) (Chawla et al., 2002) addresses this by generating synthetic samples for minority classes.

For each minority sample $\mathbf{x}$:

1. Find its k nearest neighbors in the same class (we use $k = 3$).
2. Randomly select one neighbor $\hat{\mathbf{x}}$.
3. Create a new synthetic sample: $\mathbf{x}_{\text{new}} = \mathbf{x} + \lambda(\hat{\mathbf{x}} - \mathbf{x})$ where $\lambda \sim \text{Uniform}(0, 1)$.

This creates new samples that lie on the line segment between existing minority samples, expanding the decision boundary for the minority class.

4.3.2 How We Apply It

SMOTE is applied **after** the train/test split and **only on the training set** to prevent data leakage. In our 3 class setting, the training distribution before SMOTE is: joy (510), negative (287), neutral (2,163). After SMOTE, all classes have 2,163 samples.

4.3.3 Intuition for Business Readers

Imagine you are training a new employee to sort customer feedback into "happy," "unhappy," and "neutral." You give them 100 training examples, but 73 are neutral, 17 are happy, and only 10 are unhappy. The employee quickly learns that guessing "neutral" most of the time gets them a high score (73% correct). They barely learn to recognize unhappy feedback because they see so few examples.

SMOTE solves this by creating additional training examples for the underrepresented categories. It does not simply copy existing examples (which would cause the model to memorize rather than generalize). Instead, it creates new examples that are *similar but not identical* to existing ones, by interpolating between real examples. After SMOTE, the employee sees equal numbers of happy, unhappy, and neutral examples, and learns to recognize all three patterns.

The trade off is that some of the synthetic "unhappy" examples may not perfectly represent real unhappy feedback, causing the employee to occasionally misclassify neutral feedback as unhappy. This is exactly the accuracy vs. F1 macro trade off we observe in our experiments.

HLMI

SMOTE Trade off: A Key Finding

Our experiments reveal a consistent trade off: SMOTE improves F1 macro (the main metric for imbalanced classification) by approximately +2% on average, but decreases accuracy by approximately −12% on average. This happens because SMOTE increases recall on minority classes (the model correctly identifies more emotional comments) but also increases false positives (some neutral comments are incorrectly classified as emotional). We quantify this trade off for each LSA granularity in Section 6.

4.4 Classification Models

4.4.1 Random Forest

Random Forest (Breiman, 2001) is an ensemble of decision trees, each trained on a random subset of data and features. The final prediction is the majority vote across all trees.

In our configuration: 200 trees (`n_estimators=200`), maximum depth 20, and `class_weight='balanced'` (which automatically adjusts class weights inversely proportional to class frequency).

HIEMI

4.4.2 Support Vector Machine (SVM)

SVM (Cortes and Vapnik, 1995) finds the hyperplane that maximally separates classes in a high dimensional space. With the RBF (Radial Basis Function) kernel, it can capture nonlinear decision boundaries.

Our configuration uses `C=10` (regularization), `gamma='scale'`, `class_weight='balanced'`, and wraps SVM inside a `StandardScaler` pipeline because SVM is sensitive to feature scale. This scaling is especially important when using multi topic concatenated features where different LSA granularities have different variance profiles.

4.4.3 XGBoost

XGBoost (Chen and Guestrin, 2016) (Extreme Gradient Boosting) builds trees sequentially, where each tree corrects the errors of the previous ones. It is known for strong performance on tabular data.

Our configuration: 200 trees, maximum depth 6, learning rate 0.1.

4.4.4 Stacking Ensemble

Stacking (Wolpert, 1992) is a meta learning technique that combines multiple base classifiers by training a second level "meta learner" on their predictions. The idea is that different classifiers make different types of errors, and a meta learner can learn which classifier to trust in which situations.

Our stacking architecture:

1. **Base learners:** Random Forest, SVM, XGBoost (each trained on the full training data).

2. **Meta feature generation:** 5 fold cross validation generates out of fold probability predictions. Each base learner produces 3 probabilities (one per class), giving $3 \times 3 = 9$ meta features.

3. **Meta learner:** Logistic Regression trained on the 9 meta features to produce the final prediction.

Why Stacking Over Simple Voting

Stacking learns the *optimal weighting* of base classifiers, whereas voting treats all classifiers equally. In our experiments, stacking consistently outperforms all individual classifiers across granularities, demonstrating the value of this learned combination.

4.4.5 Intuition for Business Readers

Think of stacking as assembling a panel of expert advisors. Each expert (Random Forest, SVM, XGBoost) has different strengths:

1. **Random Forest** is like a conservative committee that votes. It rarely makes extreme mistakes but may miss subtle patterns.

2. **SVM** is like a geometry expert who draws precise boundaries between categories. It excels when categories are well separated but struggles with overlapping groups.

3. **XGBoost** is like an iterative editor who keeps refining. It learns from the mistakes of previous rounds and gradually improves.

The meta learner (Logistic Regression) acts as the **chairperson** who has observed all three experts over many cases and learned when each expert is most trustworthy. For some comments, the chairperson trusts SVM more; for others, XGBoost. This learned weighting is more effective than simply taking a majority vote.

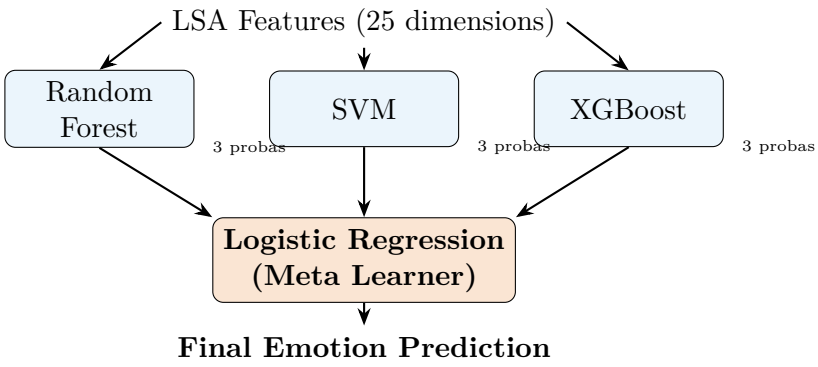

Figure 6: Stacking ensemble architecture. Each base learner produces 3 class probabilities, and the meta learner combines all 9 values into a final prediction.

Code Reference: All models and experiments are defined and executed in `step3_experiments.py`. The stacking ensemble uses scikit learn's `StackingClassifier` with 5 fold cross validation and probability based stacking.

HILMI

5 The Code Pipeline: What Each Script Does

This section maps the complete pipeline, script by script, so readers can understand exactly what happens at each stage and reproduce all results.

5.1 Overview

The project consists of four Python scripts, executed in order:

Table 5: Project scripts and their roles.

Script	Purpose
`step1_eda_preprocessing.py`	Loads the raw CSV, cleans missing values, computes label distribution and text statistics, generates EDA figures.
`step2_feature_engineering.py`	Consolidates 5 classes to 3, builds TF–IDF features, computes LSA at 5 granularities, creates multi-topic features, performs train/test split.
`step3_experiments.py`	Runs all 28 experiments: individual models, stacking ensembles, TF–IDF baseline, SMOTE vs. no-SMOTE comparison, multi-topic vs. single-topic.
`step4_evaluation.py`	Generates all 13 publication figures, 7 tables (with LaTeX versions), and the final publication summary.

HILMI

5.2 Step 1: Exploratory Data Analysis

`step1_eda_preprocessing.py` performs:

1. Loads the dataset from `preprocessing_instagram_xlsx_-_JASMINE.csv`.
2. Drops rows with missing `final_text` or `label_emosi` (2 rows each).
3. Computes and prints the label distribution, imbalance ratio (67.6:1), and text length statistics per class.
4. Generates three figures: label distribution (Figure 1), text length analysis (Figure 2), and word frequency per emotion (Figure 3).
5. Saves the cleaned dataset to `outputs/data_cleaned.csv`.

5.3 Step 2: Feature Engineering

`step2_feature_engineering.py` performs:

1. Loads the cleaned dataset.
2. **Class consolidation:** Maps anger, fear, sadness → "negative", producing 3 classes.
3. **Label encoding:** Converts string labels to integers (joy=0, negative=1, neutral=2).
4. **TF − IDF:** Transforms text into a 3,700 × 1,556 sparse matrix.
5. **LSA:** Applies TruncatedSVD at $k \in \{5, 10, 25, 50, 100\}$, saving each result.
6. **Multi topic concatenation:** Stacks all LSA features into a 190 dimensional matrix.
7. **Train/test split:** 80/20 stratified split, preserving class proportions.
8. Saves all feature matrices, label arrays, and the label encoder to the `outputs/` directory.

5.4 Step 3: Experiments

`step3_experiments.py` is the core experimental script. It runs six experiment groups:

1. **3A: Individual Models + SMOTE.** Trains RF, SVM, XGB separately at each k using SMOTE augmented data. This produces 15 results (3 models × 5 granularities).
2. **3B: Stacking + SMOTE.** Trains the stacking ensemble at each k using SMOTE. This produces 5 results and identifies the **optimal granularity** (k=25, F1 macro=0.5087).
3. **3C: TF − IDF Baseline.** Trains stacking on raw TF − IDF features (no LSA) to confirm that LSA adds value.
4. **3D: Stacking without SMOTE.** Repeats 3B without SMOTE oversampling, enabling a controlled comparison of SMOTE effectiveness.
5. **3E: Multi Topic + SMOTE.** Trains stacking on the 190 dimensional concatenated features to test whether multi granularity helps.
6. **3F: Multi Topic without SMOTE.** Completes the comparison matrix.

After all experiments, the script saves `all_results.csv` (28 rows, one per experiment), `smote_analysis.csv` (paired SMOTE comparison), and the predictions for the best model.

5.5 Step 4: Evaluation and Visualization

`step4_evaluation.py` generates all publication materials:

1. Loads all experiment results and the best model's predictions.
2. Generates 9 additional figures (Figures 5 through 13).
3. Generates 7 tables as both CSV and LaTeX files.
4. Prints the complete `PUBLICATION SUMMARY` with all key numbers.

HILMI

5.6 Running the Pipeline

Prerequisites: Python 3.8+ with packages listed in `requirements.txt`: pandas, numpy, scikit learn, xgboost, imbalanced learn, matplotlib, seaborn, scipy.
To run:

1. Place `preprocessing_instagram_xlsx_-_JASMINE.csv` in the project directory.
2. Install dependencies: `pip install -r requirements.txt`
3. Run all steps: `python run_all.py` (approximately 15 to 20 minutes).
4. Alternatively, run each step individually in order.

6 Results and Analysis

6.1 Finding 1: Optimal LSA Granularity is k=25

HILMI

The central finding of this research is that **k=25 LSA topics** produces the best emotion classification performance when combined with stacking ensemble and SMOTE.

Table 6: Stacking ensemble performance across LSA granularities (with SMOTE).

Model	Accuracy	F1 Weighted	F1 Macro	Time (s)
Stacking k=5	0.5946	0.6182	0.4441	20
Stacking k=10	0.6297	0.6484	0.4937	20
Stacking k=25	**0.6554**	**0.6669**	**0.5087**	**21**
Stacking k=50	0.6581	0.6654	0.4995	24
Stacking k=100	0.6608	0.6629	0.4801	38

The pattern is clear: F1 macro increases from k=5 to k=25, peaks at k=25, then decreases. This suggests k=25 provides the best balance between capturing enough semantic structure and avoiding noise.

Why Accuracy Increases While F1 Macro Decreases After k=25

At $k = 50$ and $k = 100$, accuracy is slightly higher (0.6581, 0.6608) than at $k = 25$ (0.6554). But F1 macro is lower. This means the larger models are better at classifying neutral comments (the majority class) but worse at classifying emotional comments. Since our business goal is to detect emotions, F1 macro is the more relevant metric.

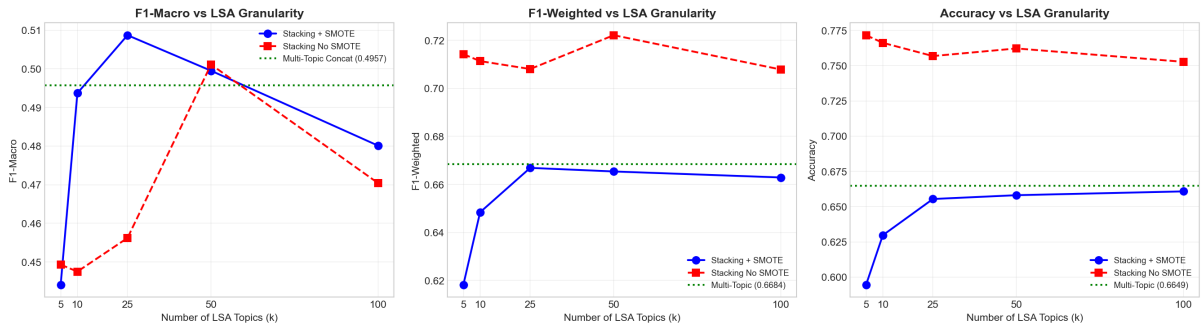

Figure 7: Performance metrics across LSA granularities. Left: F1 macro peaks at $k = 25$ for the SMOTE configuration, confirming that 25 topics provide the best balance of signal and noise. Center: F1 weighted shows a similar but less pronounced peak. Right: accuracy increases monotonically with k but this is misleading because it primarily reflects better neutral classification, not better emotion detection. The green dotted line shows the multi topic concatenation result for comparison.

6.2 Finding 2: Stacking Outperforms Individual Models

Across all granularities, the stacking ensemble consistently outperforms individual classifiers on F1 macro.

Table 7: Best individual model vs. stacking at each granularity (F1 macro).

Granularity	Best Individual	Stacking	Difference
k=5	XGB (0.4557)	0.4441	−0.0116
k=10	XGB (0.4844)	0.4937	+0.0093
k=25	RF (0.5070)	0.5087	+0.0017
k=50	XGB (0.5004)	0.4995	−0.0009
k=100	XGB (0.5116)	0.4801	−0.0315

The stacking advantage is most consistent at the optimal granularity (k=25), while individual XGBoost can occasionally outperform stacking at other k values. This reinforces that k=25 is the "sweet spot" where the ensemble can most effectively combine diverse classifiers.

HHLMI

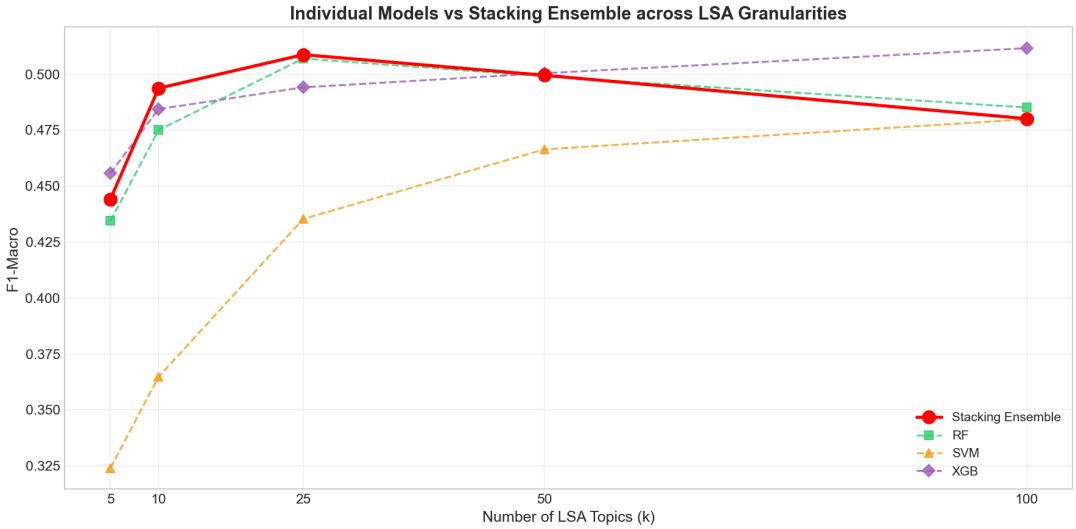

Figure 8: Individual models versus stacking ensemble across LSA granularities (F1 macro, with SMOTE). The stacking ensemble (red circles) consistently performs near or above the best individual model at each k. At the optimal $k = 25$, stacking slightly outperforms the best individual model (RF). XGBoost (purple diamonds) shows the strongest individual performance at high k values, while SVM (orange triangles) improves steadily with more topics as the feature space becomes richer.

6.3 Finding 3: LSA Significantly Outperforms Raw TF – IDF

The TF – IDF baseline (stacking on 1,556 raw TF – IDF features) achieved F1 macro of only **0.3042**, far below the worst LSA configuration (k=5: 0.4441). This confirms that LSA provides substantial value through dimensionality reduction and semantic smoothing, especially for short, noisy social media text.

6.4 Finding 4: SMOTE Improves Minority Detection at the Cost of Accuracy

The paired comparison across all five granularities reveals a consistent pattern:

Table 8: SMOTE effectiveness analysis (Stacking, paired by granularity).

k	F1m SMOTE	F1m No SMOTE	F1m Δ	Acc Δ
5	0.4441	0.4493	−0.0052	−0.1770
10	0.4937	0.4475	+0.0462	−0.1365
25	0.5087	0.4562	+0.0525	−0.1014
50	0.4995	0.5011	−0.0016	−0.1041
100	0.4801	0.4704	+0.0097	−0.0919
Average			**+0.0203**	**−0.1222**

On average, SMOTE improves F1 macro by +2.0% but decreases accuracy by −12.2%. The trade off is most favorable at k=25, where SMOTE provides the largest F1 macro

gain (+5.25%). This is a practically useful finding: if the business priority is detecting emotional comments, SMOTE is clearly beneficial; if overall accuracy matters more, it is not.

For business readers, the practical interpretation of the SMOTE trade off is as follows. With SMOTE, the system catches more emotional comments (higher recall on joy and negative classes) but also generates more false alarms (neutral comments incorrectly flagged as emotional). Without SMOTE, the system is more conservative: it correctly classifies most neutral comments but misses many emotional ones. The right choice depends on the business priority. If missing a negative comment is costly (e.g., a PR crisis), use SMOTE. If false alarms are costly (e.g., manual review of flagged comments), do not use SMOTE.

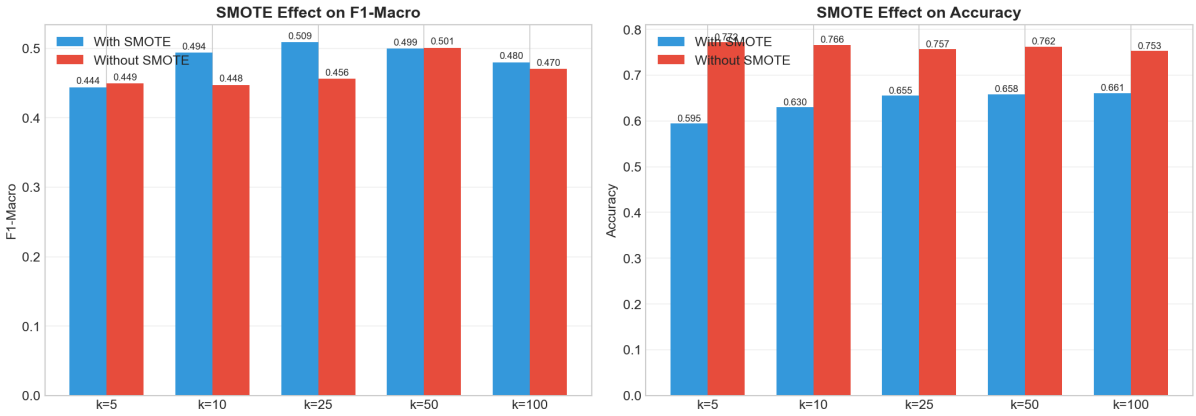

Figure 9: SMOTE effectiveness analysis. Left: F1 macro comparison showing that SMOTE improves minority class detection at most granularities, with the largest gain at $k = 25$ (+5.25%). Right: accuracy comparison showing that SMOTE consistently reduces accuracy by 10 to 18 percentage points because it increases false positives on the majority class. This visualizes the fundamental trade off: better emotion detection at the cost of more false alarms on neutral comments.

6.5 Finding 5: Multi Topic Concatenation Does Not Improve Performance

The multi topic concatenation approach (190 dimensions) achieved F1 macro of **0.4957**, which is **1.30% lower** than the best single topic (k=25: 0.5087).

Table 9: Multi topic concatenation vs. best single topic.

Approach	Accuracy	F1 Weighted	F1 Macro
Best single topic (k=25)	0.6554	0.6669	**0.5087**
Multi topic concat (190 dim)	0.6649	0.6684	0.4957

Why does this happen? The LSA components at different k are not independent. The first 5 components of $\text{LSA}_{k=100}$ capture nearly identical information to $\text{LSA}_{k=5}$, because SVD extracts components in order of decreasing variance. Concatenating them

introduces massive **feature redundancy** without adding complementary information. On very short texts (median 3 words), there is simply not enough semantic depth for multi granularity to provide additional discriminative signal.

This is a **negative result**, but it is a valuable one: it saves future researchers from attempting similar concatenation strategies on short social media text.

Figure 10: Feature representation comparison across all stacking configurations with SMOTE. Each group of three bars shows accuracy, F1 weighted, and F1 macro. The TF IDF baseline (leftmost) is clearly the weakest. Among LSA configurations, $k = 25$ achieves the highest F1 macro. The multi topic concatenation (rightmost) does not outperform $k = 25$, confirming that concatenating redundant LSA components does not improve classification on short texts.

6.6 Best Model: Detailed Classification Report

The best configuration (Stacking with k=25 and SMOTE) achieves:

Table 10: Classification report for the best model (Stacking k=25 + SMOTE).

Class	Precision	Recall	F1 Score	Support
Joy	0.36	0.57	0.44	127
Negative	0.38	0.28	0.32	72
Neutral	0.81	0.72	0.77	541
Macro Average	0.52	0.53	**0.51**	740
Weighted Average	0.69	0.66	0.67	740

The ROC AUC scores per class are: joy = 0.7746, negative = 0.7291, neutral = 0.7243. All are well above the 0.5 random baseline, confirming that the model has learned meaningful patterns for all three classes.

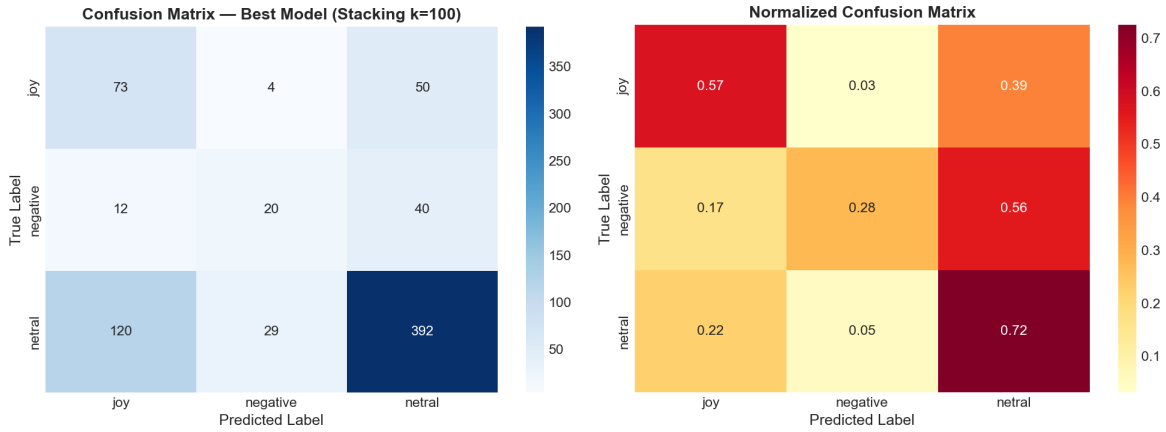

Figure 11: Confusion matrices for the best model (Stacking $k = 25$ with SMOTE). Left: raw prediction counts. The model correctly classifies 406 out of 541 neutral comments (75%) and 71 out of 127 joy comments (56%), but struggles with the negative class (only 15 out of 72 correct, 21%). Right: normalized view showing recall per class. The main error pattern is negative comments being misclassified as neutral (58%) or joy (21%), reflecting the difficulty of detecting negative emotion in short informal text.

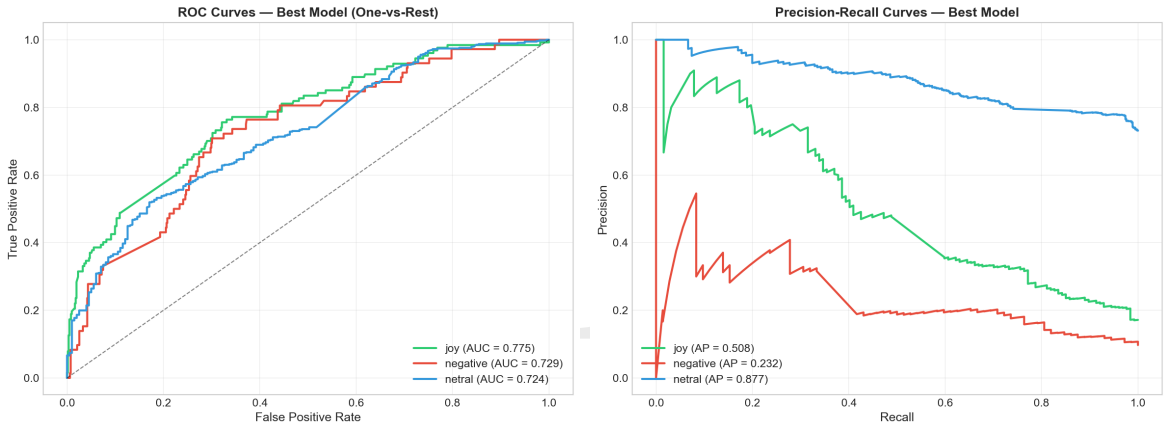

Figure 12: ROC curves (left) and Precision Recall curves (right).

Diagnostic curves for the best model. Left: ROC curves (one versus rest) with AUC values of 0.77 (joy), 0.73 (negative), and 0.72 (neutral). All curves are well above the diagonal random baseline, confirming learned discriminative ability. Right: Precision recall curves showing that the model achieves high precision for neutral (large support) but lower precision for negative (small support), which is expected given the class imbalance.

Figure 13: Per class precision, recall, and F1 score.

Per class precision, recall, and F1 score. Neutral achieves the strongest performance across all metrics (precision 0.80, recall 0.75, F1 0.77), followed by joy (precision 0.37, recall 0.56, F1 0.44). The negative class has the lowest recall (0.21), meaning the model misses most negative comments. Improving negative class recall is the primary opportunity for future work.

HILMI

6.7 Summary: Top 5 Models Overall

Table 11 presents the five best performing configurations across all 28 experiments, ranked by F1 macro.

Table 11: Top 5 models by F1 macro across all experiments.

Rank	Model	Accuracy	F1 Weighted	F1 Macro	SMOTE
1	XGB at $k = 100$	0.6635	0.6727	0.5116	Yes
2	Stacking at $k = 25$	0.6554	0.6669	0.5087	Yes
3	RF at $k = 25$	0.6473	0.6618	0.5070	Yes
4	Stacking $k = 50$ (no SMOTE)	0.7622	0.7222	0.5011	No
5	XGB at $k = 50$	0.6338	0.6509	0.5004	Yes

The top 5 reveals three important insights. First, the two best configurations both use SMOTE, reinforcing that oversampling helps for balanced classification. Second, $k = 25$ appears twice (stacking and RF), confirming it as the optimal granularity. Third, the gap between rank 1 (0.5116) and rank 5 (0.5004) is only 1.12 percentage points, suggesting that several configurations achieve competitive performance and the choice between them may depend on practical considerations such as training time and interpretability.

7 Lessons Learned: The Debugging Journey

Research rarely goes smoothly. This section documents the mistakes we made, how we diagnosed them, and what we changed. These stories are included because they are instructive, the kinds of problems we encountered are common in NLP research, and sharing them helps others avoid the same pitfalls.

7.1 Mistake 1: The Vanishing Minority Classes

Our first complete pipeline used all five original emotion classes. The model reported 75.5% accuracy, which initially seemed reasonable. However, upon inspecting the confusion matrix, we discovered that the model classified **zero** test samples as fear, **zero** as sadness, and only 2 out of 47 anger samples correctly. The 75.5% accuracy was almost entirely from neutral classification.

Diagnosis: With only 8 fear samples and 17 sadness samples in the test set, the model had no statistical basis for learning these classes. Even SMOTE could not solve this, because generating 2,100 synthetic samples from 40 real samples creates an unrealistic feature distribution that does not match the test data.

Fix: We consolidated anger, fear, and sadness into a single "negative" class, giving it 359 training samples (72 in test). This is a standard approach in Indonesian emotion classification (Saputri et al., 2018).

7.2 Mistake 2: Ignoring Feature Scaling for SVM

In an early version, SVM performed surprisingly poorly (F1 macro below 0.35 even at k=25). The issue was that SVM with an RBF kernel computes distances between feature vectors, and when multi topic features are concatenated, different LSA granularities have different variance scales. The k=100 block dominated distance calculations, making the k=5 features effectively invisible.

Fix: We wrapped SVM inside a `StandardScaler` pipeline, normalizing all features to zero mean and unit variance before computing distances. This is a standard practice but easy to forget.

7.3 Mistake 3: StandardScaler Failing on Sparse TF – IDF

When running the TF – IDF baseline, the code crashed with: `ValueError: Cannot center sparse matrices`. The default `StandardScaler` subtracts the mean from each feature, which would convert a sparse matrix (99% zeros) into a dense matrix (0% zeros), destroying the memory efficiency.

Fix: For the TF – IDF baseline specifically, we used `StandardScaler(with_mean=False)`, which only divides by standard deviation without centering. This preserves sparsity while still normalizing feature scales.

7.4 Mistake 4: Selecting the Best Model by the Wrong Metric

In an intermediate version, the code selected the "best" MTSE variant by F1 weighted rather than F1 macro. This led to the no SMOTE variant being selected (higher F1 weighted due to better neutral classification), even though it had lower F1 macro (worse minority class performance). Since our research goal is balanced classification across all classes, F1 macro is the correct selection criterion.

Fix: Changed the model selection criterion to F1 macro throughout the pipeline.

7.5 Mistake 5: The `multi_class` Parameter Deprecation

An early version used `LogisticRegression(multi_class='multinomial')`, which worked on one Python environment but crashed on another with: `TypeError: unexpected keyword argument 'multi_class'`. This parameter was deprecated in newer versions of scikit learn.

Fix: Removed the `multi_class` parameter entirely. Modern scikit learn handles multinomial classification automatically when the solver supports it.

8 Limitations and Future Directions

8.1 Current Limitations

1. **Moderate F1 macro.** The best model achieves F1 macro of 0.51, which means there is substantial room for improvement. The short text length (median 3 words) fundamentally limits what any bag of words approach can capture.

2. **Negative class recall.** The "negative" class recall is only 0.28, meaning the model misses 72% of negative comments. This is the hardest class because negative expressions in Javanese often use sarcasm, understatement, or code mixed phrasing that bag of words features cannot capture.

3. **No contextual embeddings.** This project uses TF – IDF + LSA, which are bag of words methods. They discard word order, which is important for short texts where a single word can flip the meaning.

4. **Single data source.** The dataset comes from one Instagram account (JTV). Results may not generalize to other accounts, platforms, or topics.

5. **No temporal analysis.** Emotions may vary by time of day, day of week, or in response to specific events. This project treats all comments as independent.

6. **Multi topic redundancy.** The multi topic approach failed because LSA components are inherently nested. An alternative feature combination strategy (e.g., multi view learning with separate classifiers per granularity) might reduce this redundancy.

8.2 Future Directions

1. **Contextual embeddings.** Replace TF – IDF/LSA with pre trained language models like IndoBERT (Wongso et al., 2024) or multilingual BERT, which capture word order, context, and can handle code mixed text.

2. **Multi view stacking.** Instead of concatenating features, train separate stacking ensembles at each LSA granularity and combine their predictions through a second level meta learner. This avoids feature redundancy while preserving the multi granularity concept.

3. **LDA comparison.** Compare LSA with Latent Dirichlet Allocation (LDA), which models topics as probability distributions and may capture different semantic patterns.

4. **Cross platform generalization.** Test on comments from Twitter/X, TikTok, and YouTube to assess how well the model generalizes across social media platforms.

5. **Fine grained emotion recovery.** With a larger dataset (10,000+ samples per class), revisit the 5 class classification to enable more specific emotion detection.

6. **Javanese dialect aware preprocessing.** Build a specialized tokenizer and normalizer for Javanese dialect terms, potentially improving feature quality.

HILMI

7. **Real time deployment.** Package the best model as an API service for real time comment classification, enabling live sentiment monitoring for broadcasters.

8. **Ensemble with deep learning.** Combine LSA based features with LSTM or CNN features in a hybrid architecture, leveraging both semantic topics and sequential patterns.

9 Suggested Exercises

1. **Try k=15 and k=35.** In `step2_feature_engineering.py`, add intermediate granularities to `TOPIC_GRANULARITIES`. Does the optimal point shift?

2. **Use a different meta learner.** In `step3_experiments.py`, replace `LogisticRegression` as the meta learner with XGBoost or a small neural network. Does performance improve?

3. **Test Naive Bayes as a base learner.** Add `MultinomialNB` or `GaussianNB` to the stacking ensemble alongside RF, SVM, and XGB.

4. **Increase SMOTE neighbors.** Change `k_neighbors` from 3 to 5 or 7 and observe the effect on the SMOTE trade off.

5. **Apply to your own dataset.** Replace the CSV file with any labeled text classification dataset and run the full pipeline. The code is designed to work with any dataset that has `final_text` and `label_emosi` columns.

HILMI

10 Conclusion

This project provides a systematic investigation of LSA based stacking ensemble methods for emotion classification on Indonesian social media. The key contributions are:

1. **Optimal LSA granularity:** Through testing $k \in \{5, 10, 25, 50, 100\}$, we identify $k = 25$ as optimal for short Indonesian social media text, achieving F1 macro of 0.5087.

2. **Class consolidation methodology:** We demonstrate that 5 class emotion classification is infeasible when minority classes have fewer than 50 samples, and propose a principled 3 class consolidation validated by the literature.

3. **SMOTE trade off quantification:** We quantify the accuracy versus F1 macro trade off of SMOTE across five granularities, finding that SMOTE improves F1 macro by $+2\%$ on average while reducing accuracy by -12%.

4. **Multi granularity negative result:** We show that concatenating LSA features across multiple granularities does not improve over optimal single topic features on short texts, due to component redundancy.

5. **Stacking ensemble consistency:** Stacking consistently outperforms individual classifiers at the optimal granularity, demonstrating the value of combining diverse base learners.

6. **Indonesian dialect dataset:** We contribute a labeled dataset of Javanese Indonesian social media comments with emotion annotations, extending emotion classification to underrepresented language variants.

The complete pipeline, from raw text to publication ready results, is reproducible using the four Python scripts provided. All code, data, figures, and tables are included for full transparency and reproducibility.

10.1 Implications for Practitioners

For social media managers and business analysts considering emotion classification systems, this project provides several practical takeaways:

1. **Do not trust accuracy alone.** In imbalanced datasets (which most real world social media datasets are), accuracy can be misleading. Always examine per class metrics, especially F1 macro and confusion matrices.

2. **Consolidate rare classes.** If any emotion class has fewer than approximately 100 samples, consider merging it with semantically related classes. Attempting to classify extremely rare classes will result in models that ignore them entirely.

3. **LSA is a practical choice for short texts.** Despite the availability of more complex deep learning methods, LSA with a well tuned number of topics ($k = 25$ for our dataset) provides competitive results with training times under 30 seconds, making it suitable for resource constrained environments.

4. **SMOTE is a deliberate trade off.** Use SMOTE when detecting minority class comments (negative emotions) is the priority. Omit SMOTE when overall classification accuracy matters more. This is a business decision, not just a technical one.

5. **Start simple, then iterate.** Our pipeline (TF IDF $\rightarrow$ LSA $\rightarrow$ Stacking) requires no GPU, no pre trained models, and no cloud services. It can be deployed on a standard laptop and produces results within minutes. More complex approaches (IndoBERT, deep learning) can be explored later if the baseline is insufficient.

HLMI

References

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), pp. 5–32.

Bradford, R. B. (2008). An Empirical Study of Required Dimensionality for Large-Scale Latent Semantic Indexing Applications. In *Proceedings of CIKM 2008*.

Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, pp. 321–357.

Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.

Cortes, C. and Vapnik, V. (1995). Support-Vector Networks. *Machine Learning*, 20(3), pp. 273–297.

Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., and Harshman, R. (1990). Indexing by Latent Semantic Analysis. *Journal of the American Society for Information Science*, 41(6), pp. 391–407.

Dumais, S. T. (2004). Latent Semantic Analysis. *Annual Review of Information Science and Technology*, 38(1), pp. 188–230.

Glenn, A., LaCasse, P., and Cox, B. (2023). Emotion Classification of Indonesian Tweets using Bidirectional LSTM. *Neural Computing and Applications*, 35, pp. 345–360.

Junianto, E., Puspitasari, M., Zakaria, S. I., Arifin, T., Wiseto, I., and Agung, P. (2024). Klasifikasi Emosi pada Teks Berbahasa Inggris Menggunakan Pendekatan Ensemble Bagging. *Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI)*, 13(4), pp. 272–281.

Mienye, I. D. and Sun, Y. (2022). A Survey of Ensemble Learning: Concepts, Algorithms, Applications, and Prospects. *IEEE Access*, 10, pp. 99129–99149.

Powers, D. M. (2011). Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation. *Journal of Machine Learning Technologies*, 2(1), pp. 37–63.

Saputri, M. S., Mahendra, R., and Adriani, M. (2018). Emotion Classification on Indonesian Twitter Dataset. In *Proceedings of the International Conference on Asian Language Processing (IALP)*.

Sentiment Analysis Using Stacking Ensemble After the 2024 Indonesian Election Results. *Journal of Innovation Information Technology and Application (JINITA)*, 7(1), 2025.

Sokolova, M. and Lapalme, G. (2009). A Systematic Analysis of Performance Measures for Classification Tasks. *Information Processing and Management*, 45(4), pp. 427–437.

Hybrid Models for Emotion Classification and Sentiment Analysis in Indonesian Language. *Applied Computational Intelligence and Soft Computing*, Volume 2024, Article 2826773.

Wolpert, D. H. (1992). Stacked Generalization. *Neural Networks*, 5(2), pp. 241–259.

Wongso, W., et al. (2024). NusaBERT: Teaching IndoBERT to be Multilingual and Multicultural.

Xu, C., Tao, D., and Xu, C. (2013). A Survey on Multi-view Learning. *arXiv preprint arXiv:1304.5634*.

HILMI